MODULE 21 · BLOCK 4 - FUNCTIONS AND EXCEPTIONS

Handling exceptions

try and except, the order of except branches, else and finally, raise, and exceptions travelling through function calls.

BLOCK 4 of 4 EXAM WEIGHT 28% MODULE 21 of 28

admin@sthithapragna.com

When an exception occurs in a try block, the rest of the block is skipped, and Python runs the **first** except branch that matches. **At most one** branch runs.

The try block stops at the first exception

```
1  try:
2      print("start")
3      print(1 / 0)            # raises here
4      print("never")          # skipped
5  except ZeroDivisionError:
6      print("caught")
7  print("after")              # start, caught, after
```

1-4 Put code that might fail in the try block. Everything after the failing line is skipped.

5-6 The except branch handles the exception, and the program carries on after it.

The first matching branch wins, and parents match

```
1  try:
2      print(10 // 0)
3  except ArithmeticError:        # matches: ZeroDivisionError is a kind of it
4      print("A")
5  except ZeroDivisionError:      # never reached
6      print("Z")
7  except:
8      print("?")                  # output: A
```

3 A branch matches the exception itself, or any of its parents.

5 Once a branch has matched, the others are skipped, even a more exact one.

Broad first hides narrow. A bare except first is illegal

BROAD BEFORE NARROW

Legal, but the narrow branch is dead

```
try:
    raise Exception
except BaseException:
    print("1")
except Exception:
    print("2")
```

Prints 1. The Exception branch can never run, but this is not an error.

BARE EXCEPT FIRST

SyntaxError

```
try:
    raise ValueError
except:
    print("x")
except ValueError:
    print("v")
```

SyntaxError: default 'except:' must be last. Nothing runs.

THE RULE

Order branches from the most specific to the most general, with any bare except: last.

A tuple of exceptions in one except

```
1  for v in ["x", "0", "4"]:
2      try:
3          print(10 / int(v))
4      except (ValueError, ZeroDivisionError) as e:
5          print("problem:", e)
```

4 Either exception is handled by this one branch. as e names the exception object.

5 Printing e shows the exception's message.

Output: problem: invalid literal for int() with base 10: 'x', then problem: division by zero, then 2.5

Brackets around several exceptions became optional

PYTHON 3.13 AND EARLIER

SyntaxError

```
1  try:
2      print(1 / 0)
3  except ZeroDivisionError, ValueError:
4      print("caught")
```

SyntaxError: multiple exception types must be parenthesized.

PYTHON 3.14 (PEP 758)

Legal, without as

```
1  try:
2      print(1 / 0)
3  except ZeroDivisionError, ValueError:
4      print("caught")
```

Prints caught. With as e, brackets are still required, even on 3.14.

ON THE EXAM

Exam answer: the brackets are required. Choose the bracketed option; it is correct in every version.

else runs if nothing went wrong. finally always runs

```
1  try:
2      x = int("5")
3  except ValueError:
4      print("E", end="")
5  else:
6      print("L", end="")          # no exception, so else runs
7  finally:
8      print("F", end="")          # runs in every case
9  print("!")                     # output: LF!
```

5-6 else runs only when the try block finished without an exception.

7-8 finally runs last, whatever happened: exception, no exception, even a return.

The order is fixed: try, except, else, finally. else needs at least one except; try with only finally is legal.

finally runs even when a branch returns

```
 1  def f():
 2      try:
 3          print("a", end="")
 4          x = [1][3]              # IndexError
 5          print("b", end="")      # skipped
 6      except IndexError:
 7          print("c", end="")
 8          return
 9      finally:
10          print("d", end="")      # still runs, before the return completes
11      print("e", end="")          # never reached
12  f()                            # output: acd
```

8-10 The return is held up until finally has run.

11 After the return, the rest of the function is skipped.

return inside finally now gives a warning

- A **return**, **break** or **continue** that leaves a finally block silently discards any exception, or overrides the earlier return.
- Python 3.14 (PEP 765) reports this with **SyntaxWarning: 'return' in a 'finally' block**.
- It is a warning, not an error. The code still runs exactly as before.

> ON THE EXAM
>
> try: return 1, finally: return 2 still returns 2. Only the warning is new.

raise signals a problem your code has detected

```
1  def check(age):
2      if age < 0:
3          raise ValueError("age cannot be negative")
4      return age
5  try:
6      check(-1)
7  except ValueError as e:
8      print("error:", e)        # error: age cannot be negative
```

3 raise creates the exception, with an optional message, and starts it on its way.

6-8 It is handled exactly like a built-in one.

In Python terminology, exceptions are raised, not thrown. raise ValueError, without brackets, also works.

An unhandled exception moves up to the caller

```
1  def half(n):
2      return 10 / n            # no try here: the exception leaves half
3  def safe(n):
4      try:
5          return half(n)       # ... and arrives here, inside a try
6      except ZeroDivisionError:
7          return -1
8  print(safe(0), safe(4))     # -1 2.5
```

2 half does not handle the exception, so it leaves half immediately.

5-7 It arrives in safe, inside the try, so safe's except branch handles it.

Nested handlers: the inner one gets the first chance

```
1  try:
2      try:
3          [][0]                # IndexError
4      except KeyError:
5          print("inner")     # does not match
6  except IndexError:
7      print("outer")         # output: outer
```

4-5 The inner handler does not match, so the exception passes outwards.

6-7 The outer handler matches and deals with it.

Errors that except branches do not catch

SITUATION	WHAT HAPPENS
A SyntaxError anywhere in the file	nothing runs: syntax is checked before any try exists
break inside try, but not inside a loop	SyntaxError: 'break' outside loop
an exception type no branch names	it travels on, and may end the program
an exception inside an except branch	a new exception, not handled by the same try
except Exception, and the user presses Ctrl+C	KeyboardInterrupt is not an Exception: it gets through

THE TRAP

try handles exceptions raised while its block runs. It cannot rescue code that never compiled.

Words about exceptions and debugging

- Exceptions are **raised**, not thrown. They are **handled** in except branches.
- A **debugger** runs a program step by step so you can trace it precisely.
- **Print debugging** means adding print() calls to see values while a program runs.
- Running without errors does not prove code is correct: some paths may never have run.
- Python's style is to try the operation and handle failure, rather than check everything first.

What to carry into the exam

- The try block stops at the first exception. The **first matching** except branch runs; parents match.
- At most one except branch runs. A bare except: must be last, or it is a SyntaxError.
- else: no exception happened. finally: always runs, even after return.
- Several exceptions: **except (A, B):** with brackets, for the exam.
- Unhandled exceptions travel up through callers. SyntaxErrors cannot be caught.

What is the output of this program?

```
try:
    print(10 // 0)
except ArithmeticError:
    print("A")
except ZeroDivisionError:
    print("Z")
except:
    print("?")
```

A. A

B. Z

C. A, then Z

D. The code raises a SyntaxError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  try:
2      print(10 // 0)
3  except ArithmeticError:
4      print("A")
5  except ZeroDivisionError:
6      print("Z")
7  except:
8      print("?")
```

ZeroDivisionError is a kind of ArithmeticError, so the first branch matches.

A. **A**
The first branch matches through the parent, and ends the search.

B. **Z**
The exact match comes second. The first match wins.

C. **A, then Z**
At most one except branch runs.

D. **The code raises a SyntaxError**
The bare except is last, which is legal. Broad before narrow is not an error.

THE SIGNAL

Parents match. The first matching branch wins, even when a more exact one follows.

What happens when this program runs?

```
print("go")
try:
    raise ValueError
except:
    print("x")
except ValueError:
    print("v")
```

A go, then x

B go, then v

C go, then x, then v

D Nothing is printed; the program raises a SyntaxError

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  print("go")
2  try:
3      raise ValueError
4  except:
5      print("x")
6  except ValueError:
7      print("v")
```

SyntaxError: default 'except:' must be last. Found before line 1 runs.

A go, then x

What you would get if Python allowed this order. It does not.

B go, then v

The file never compiles, so nothing runs.

C go, then x, then v

Wrong twice: it does not compile, and only one branch could ever run.

D Nothing is printed; the program raises a SyntaxError

A bare except must be last, so even go is never printed.

THE SIGNAL

A bare except: that is not last is a SyntaxError, so nothing in the file runs.

The user types 4 and presses Enter.

What happens when this program runs?

```
try:
    n = input()
    print(n / 2)
except ValueError:
    print("bad value")
except ZeroDivisionError:
    print("zero")
print("done")
```

A bad value, then done

B zero, then done

C The program stops with an unhandled TypeError

D 2.0, then done

◆ PAUSE AND DECIDE

What happens when this program runs?

```
1  try:
2      n = input()
3      print(n / 2)
4  except ValueError:
5      print("bad value")
6  except ZeroDivisionError:
7      print("zero")
8  print("done")
```

n is the str "4". A str divided by an int is a TypeError, and no branch names TypeError.

A. bad value, then done
The problem is the type, str, not a bad value. That is TypeError.

B. zero, then done
Nothing divides by zero here.

C. The program stops with an unhandled TypeError
No branch matches TypeError, so it travels on and ends the program.

D. 2.0, then done
Forgets that input() returns a str. Nothing converts it.

THE SIGNAL

If no except branch matches, the try might as well not be there. Name the exception first.

What is the output of this program?

```
try:
    x = int("5")
except ValueError:
    print("E", end="")
else:
    print("L", end="")
finally:
    print("F", end="")
print("!")
```

A ELF!

B LF!

C F!

D L!

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  try:
2      x = int("5")
3  except ValueError:
4      print("E", end="")
5  else:
6      print("L", end="")
7  finally:
8      print("F", end="")
9  print("!")
```

int("5") succeeds: except skipped, else runs, finally runs.

A ELF!

Runs the except branch too. There was no exception.

B LF!

No exception: else runs, then finally, then the next line.

C F!

Forgets that else runs when the try block succeeds.

D L!

Forgets that finally always runs.

THE SIGNAL

No exception: try, else, finally. Exception handled: try, except, finally.

What is the output of this program?

```
1  def half(n):
2      return 10 / n
3
4  def safe(n):
5      try:
6          return half(n)
7      except ZeroDivisionError:
8          return -1
9
10 print(safe(0), safe(4))
```

A `-1 2.5`

B `-1 2`

C The code raises a ZeroDivisionError

D `None 2.5`

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def half(n):
2      return 10 / n
3
4  def safe(n):
5      try:
6          return half(n)
7      except ZeroDivisionError:
8          return -1
9
10 print(safe(0), safe(4))
```

The exception leaves half, arrives in safe's try, and is handled there.

A `-1 2.5`

The error travels up to safe, which handles it; 10 / 4 is 2.5.

B `-1 2`

/ always gives a float: 2.5.

C The code raises a ZeroDivisionError

half has no handler, but its caller does.

D `None 2.5`

The except branch returns -1, not None.

THE SIGNAL

An exception with no handler in its own function travels to the caller's handler.

What is the output of this program?

```
1  def f():
2      try:
3          print("a", end="")
4          x = [1][3]
5          print("b", end="")
6      except IndexError:
7          print("c", end="")
8          return
9      finally:
10         print("d", end="")
11     print("e", end="")
12
13 f()
```

A acde

B abcd

C acd

D ad

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  def f():
2      try:
3          print("a", end="")
4          x = [1][3]
5          print("b", end="")
6      except IndexError:
7          print("c", end="")
8          return
9      finally:
10         print("d", end="")
11     print("e", end="")
12
13 f()
```

a; IndexError skips b; c; return waits for finally: d; e is never reached.

A **acde**

The return in the except branch leaves f before e.

B **abcd**

The try block stops at the IndexError, so b is skipped.

C **acd**

a, then the error skips b, c, finally d, and return skips e.

D **ad**

The except branch does run and prints c.

THE SIGNAL

Trace three things: where the try block stops, which branch runs, and whether anything returns.

What is the output of this program?

```
try:
    raise KeyError("k")
except LookupError:
    print("lookup")
except KeyError:
    print("key")
```

A key

B lookup

C lookup, then key

D The code raises a KeyError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  try:
2      raise KeyError("k")
3  except LookupError:
4      print("lookup")
5  except KeyError:
6      print("key")
```

KeyError is a kind of LookupError, so the first branch matches.

A **key**
The exact match comes second, and is never reached.

B **lookup**
KeyError's parent, LookupError, matches first.

C **lookup, then key**
Only one except branch ever runs.

D **The code raises a KeyError**
It is handled, by the LookupError branch.

THE SIGNAL

Module 20's tree decides module 21's handlers: a parent branch catches all its children.

> Consider try statements in Python in general.

Which two statements are true? (Choose two.)

A At most one except branch runs for one exception

B A SyntaxError inside a try block is caught by a bare except

C except Exception: catches KeyboardInterrupt

D The finally branch runs whether or not an exception occurred

◆ PAUSE AND DECIDE

Which two statements are true? (Choose two.)

A At most one except branch runs for one exception

The first matching branch runs; the rest are skipped.

B A SyntaxError inside a try block is caught by a bare except

Syntax is checked before anything runs, so no try exists yet.

C except Exception: catches KeyboardInterrupt

KeyboardInterrupt sits beside Exception, not under it.

D The finally branch runs whether or not an exception occurred

That is what finally is for, even after a return.

THE SIGNAL

One branch at most; finally always; SyntaxErrors and KeyboardInterrupt get past the usual handlers.